Problem A. Orchard Fence

Source file name:	Orchard.c, Orchard.cpp, Orchard.java, Orchard.py
Input:	`Standard`
Output:	`Standard`

Branory and Gredon inherited an apple orchard from some distant relatives. When they visited the orchard for the first time, they were pleasantly surprised to find that all n apple trees in the orchard had trunks with exactly the same diameter d.

They decided that it was too much work to take care of all n trees, so they would only keep k of them. To keep wild animals out of the orchard, they would build a single contiguous fence enclosing the k trees they would keep. The fence may bend into any shape, but its total length must be as small as possible.

Gredon went to buy fencing material, while Branory stayed behind to uproot trees from the orchard. Gredon wants to buy the minimum amount of fencing to enclose the trunks of the k apple trees that they will keep, but he realized that he doesn't know which trees Branory will uproot! Gredon knows that Branory will randomly uproot trees until exactly k trees remain. What is the expected length of fencing that Gredon needs to buy to enclose the remaining trees?

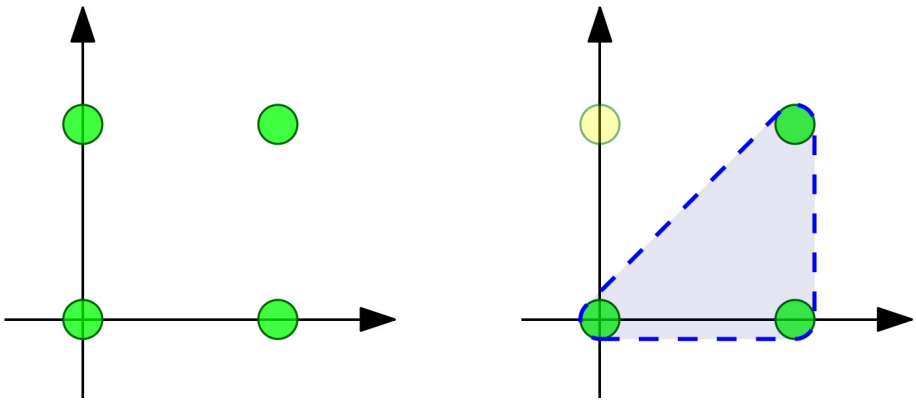

Figure 1. Illustration of the trees for Example Case 3. The figure on the right shows the optimal fence if the top-left tree is uprooted.

Input

The first line of input contains three integers n, k, and d ($1 \le k \le n \le 2000$, $1 \le d \le 100$), where n is the number of apple trees originally in the orchard, k is the number of trees that will be kept, and d is the diameter of each tree.

The next n lines each contain two real numbers x and y giving the coordinates of the center of a tree trunk. All coordinates have an absolute value no larger than 10^4 and are given with exactly two digits after the decimal point. The trunks of the trees are guaranteed not to overlap or touch.

Output

Output a single real number, the expected length of fencing that Gredon needs to buy. Your answer will be considered correct if it has a relative or absolute error of at most 10^{-6}.

Example

Input	Output
3 1 10 1.00 1.00 30.00 31.50 55.00 67.50	31.4159265359
4 3 10 -100.00 -100.00 0.00 0.00 100.00 100.00 200.00 200.00	738.5227077224
4 3 20 100.00 100.00 0.00 0.00 100.00 0.00 0.00 100.00	404.2532093091 2025

Explanation

Example Case 1: Regardless of which tree is kept, Gredon needs to buy enough fencing to enclose that tree.

Problem B. Breakout

Source file name:	Breakout.c, Breakout.cpp, Breakout.java, Breakout.py
Input:	`Standard`
Output:	`Standard`

Uh oh! After you gave a low rating to one of Merlin's riddles, he has taken revenge by locking you inside his magical dungeon! This dungeon can be modeled as a circle with n rooms arranged along its circumference. The rooms are labeled from 1 to n in clockwise order.

You start at room 1 and know that there exists an exit in one of the rooms. To get to the exit room, you can walk either clockwise or counterclockwise to a neighboring room. However, Merlin is smart. To impede your escape, Merlin has placed wooden crates in several hallways connecting two adjacent rooms. To pass through a hallway, you must break any wooden crates blocking it.

You want to make it out to the exit room as quickly as possible. What is the minimum number of crates you must break through to escape?

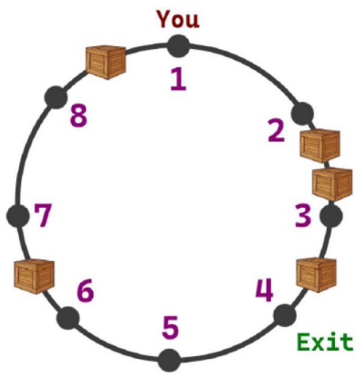

Figure 2. Illustration of the example case.

Input

The first line of input contains three integers n, x, and m ($2 \le n \le 1000,\ 1 \le x \le n,\ 0 \le m \le 1000$), meaning that there are n rooms with room x being the exit room, and there are m crates.

Each of the next m lines contains a single integer r ($1 \le r \le n$), indicating that there is a crate in the hallway clockwise between room r and the next room along the circumference. If $r = n$, it means that the crate is in the hallway between room n and room 1. These integers may contain duplicates, meaning that multiple crates are placed in the same hallway.

Output

Output a single integer, the minimum number of crates you must destroy to breakout.

Example

Input	Output
8 4 5 2 6 3 2 8	2

Problem C. One Way Only

Source file name:	Oneway.c, Oneway.cpp, Oneway.java, Oneway.py
Input:	`Standard`
Output:	`Standard`

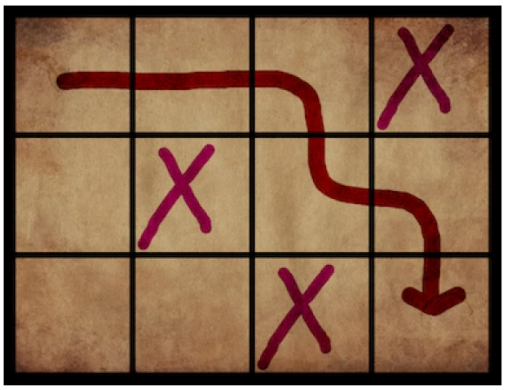

Figure 3. The preferred path in the sample case. Three tiles are X-ed out to enforce this path.

You've been tasked with mapping out one of the most dangerous hiking routes deep within the Amazon rainforest. This hiking route exists on a rectangular area of land that has been tiled into a 2D grid with r rows and c columns. Hikers who dare attempt this trail begin on the top-left tile $(1,\ 1)$ and work their way down to tile $(r,\ c)$. In order to finish as fast as possible, every hiker will move only rightward or downward during their hike.

After a thorough examination of the entire land area, you've uncovered just how unforgiving this jungle can be. With many tiles containing extreme hazards (from apex predators to mangrove swamps), you've determined that there is exactly one path through the jungle consisting of only rightward and downward moves that also avoids all of the major obstacles. You call this path your *preferred path*.

However, hikers who try this route may feel adventurous and follow a different path (still consisting of only rightward and downward moves) instead of the preferred path. To discourage the hikers from doing this, you plan to X out some tiles on the map, knowing that a hiker will not pass through an X-ed out tile under any circumstances. Consequently, if the only path from $(1,\ 1)$ to $(r,\ c)$ that does not pass through an X-ed out tile is your preferred path, then the hikers will have no choice but to follow it!

Given the map dimensions and your preferred path, can you determine the minimum number of tiles you must X out to force all hikers to take your preferred path?

Input

The first line of input contains two integers r and c ($1 \le r, c \le 10^5,\ r \cdot c \ge 2$), the number of rows and columns on the map.

The next line has a string of length $r + c - 2$ which contains exactly $r - 1$ '`D`'s and $c - 1$ '`R`'s: the sequence of moves that follows your preferred path through the jungle. A '`D`' represents a downward move on the map, while an '`R`' represents a rightward move.

Output

Output a single integer, the minimum number of tiles you must X out on the map to guarantee that all future hikers will take your preferred path through the jungle.

Example

Input	Output
3 4 RRDRD	3

Problem D. GUID Generator

Source file name:	GUID.c, GUID.cpp, GUID.java, GUID.py
Input:	`Standard`
Output:	`Standard`

Anna is preparing a new database for her web application, in which a Globally Unique Identifier (GUID) is assigned to each table row. Getting tired of the traditional random GUID generation method, Anna decides to try something new this time.

Anna finds a tree with n nodes, where between each pair of nodes there exists one unique simple path (A simple path visits each node at most once). She assigns each node one hexadecimal character (`0-9` or `a-f`). To generate a GUID, she selects two nodes s and t (not necessarily distinct), and constructs a string by concatenating all the hexadecimal characters along the simple path from s to t. This resulting hexadecimal string is then used as a GUID. Note that the GUIDs generated by this method do not have a fixed length. Besides, the GUID produced by the path from s to t may not be the same as the GUID produced by the path from t to s.

Anna can generate different GUIDs by choosing different nodes s and t. To ensure the quality of her GUID generation method, she wants to know how many unique GUIDs can be generated from the given tree.

Input

The first line of input has a single integer n ($2 \le n \le 2000$), the number of nodes in the tree. The nodes are numbered from 1 to n. The next line contains a string of n hexadecimal characters (`0-9` or `a-f`), where the ith character is the hexadecimal character assigned to node i.

The following $n-1$ lines each contain two integers u and v ($1 \le u, v \le n,\ u \ne v$), indicating that there is an edge between nodes u and v. It is guaranteed that the given edges form a tree.

Output

Output a single integer, the number of unique GUIDs that can be generated from the given tree.

Example

Input	Output
4 abab 1 2 3 2 2 4	8
6 01aa10 1 2 2 3 2 4 4 5 5 6	18

Explanation

Example Case 1: Starting from node 1 or node 3, you can obtain 4 unique GUIDs: `a`, `ab`, `aba`, and `abb`. Starting from node 2, you can obtain 3 unique GUIDs: `b`, `ba`, and `bb`. Starting from node 4, you can obtain 3 unique GUIDs: `b`, `bb`, and `bba`. In total, there are 8 unique GUIDs that can be obtained.

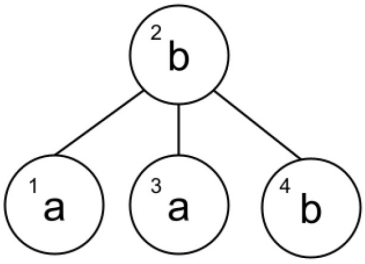

Figure 4. Illustration of the first example case. The node numbers are marked at the top-left of each node, and the hexadecimal characters are shown in the center.

Problem E. Photo Encoding

Source file name:	Encoding.c, Encoding.cpp, Encoding.java, Encoding.py
Input:	`Standard`
Output:	`Standard`

You've been tasked with printing some of your old family photos. These photos are ancient - not only are they black and white, but they are also incredibly pixelated! In fact, each photo can be represented as an $n \times n$ grid of pixels, where each pixel is either black or white.

Before you print each photo, you want to buy a frame that perfectly fits an $n \times n$ photo. However, you realize that you do not know n (the dimensions of the photo are unknown)! To make things worse, your computer stores the photo in an unreadable binary format - the only information you can recover about this photo is the list of Manhattan distances of each black pixel from the top-left pixel. The Manhattan distance between two pixels at (r_1, c_1) and (r_2, c_2) is $|r_1 - r_2| + |c_1 - c_2|$.

For example, the 5×5 photo in Figure 5 corresponds to the list $[1, 4, 4]$. You notice that there are multiple possible photos that could correspond to the same list. As an example, the 4×4 photo shown in Figure 6 also corresponds to the list $[1, 4, 4]$.

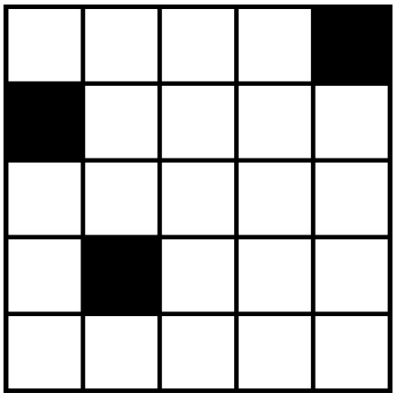

Figure 5. A 5×5 photo that corresponds to $[1, 4, 4]$.

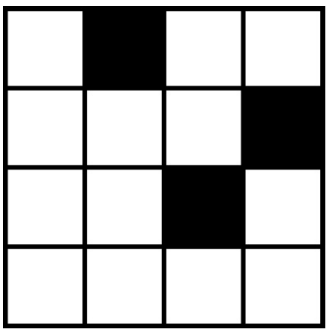

Figure 6. A 4×4 photo that also corresponds to $[1, 4, 4]$.

With this in mind, you want to be prepared when buying the picture frame. Given a list of Manhattan distances, can you determine the smallest possible n such that there exists an $n \times n$ photo corresponding to this list?

Input

The first line of input contains one integer m ($1 \leq m \leq 1000$), the number of black pixels in the photo.

Each of the next m lines contains a single integer between 0 and 200 (inclusive), representing the Manhattan distance from one black pixel to the top-left pixel of the photo. The distances are given in ascending order and are guaranteed to correspond to a valid photo.

Output

Output a single integer, the minimum side length n such that there exists an $n \times n$ photo corresponding to the input list.

Example

Input	Output
3 1 4 4	4
8 0 1 3 5 5 5 5 6	5

Problem F. Meeting Free Fridays

Source file name:	Fridays.c, Fridays.cpp, Fridays.java, Fridays.py
Input:	`Standard`
Output:	`Standard`

It's Friday! You can't wait to enjoy all t minutes of the day.

But as you look at your calendar, you see that you have n meetings scheduled, each occupying a contiguous interval of time during the day. You would like to attend as many meetings as you can, but at the same time you don't want to be stuck in meetings all day, so you decide that for at least k minutes of the day, you must not be in a meeting.

What is the maximum number of meetings you can attend in full while ensuring that at least k minutes of your day are meeting-free? You can only attend one meeting at a time, but you may attend a new meeting if it starts exactly when the previous one ends.

Input

The first line of input contains three integers n, t, and k $\left(1 \le n \le 5000,\ 1 \le k \le t \le 10^9\right)$, where n is the number of meetings, t is the number of minutes in the day, and k is the number of minutes you want to be meeting-free. Each of the next n lines contains two integers s and e $(0 \le s < e \le t)$, describing the start and end times of a meeting. The times are given in minutes since the beginning of the day.

Output

Output a single integer, the maximum number of meetings you can attend while ensuring that at least k minutes of the day are meeting-free.

Example

Input	Output
5 5 1 0 1 1 2 0 5 3 4 4 5	4
1 1000000000 1 0 1000000000	0

Problem G. Shinjuku Station

Source file name:	Shinjuku.c, Shinjuku.cpp, Shinjuku.java, Shinjuku.py
Input:	`Standard`
Output:	`Standard`

Audrey is planning her first day of vacation in Tokyo! Her first stop is a Honkai: Star Rail pop-up shop. She has heard from her friends that the trains in Japan are a lot like her favorite train, the Astral Express, so she decides to take a series of trains to the pop-up. As luck would have it, she has to transfer at Shinjuku Station, the largest train station in Tokyo.

Audrey is thinking of taking one of the n trains that will arrive at Shinjuku Station, and then transferring to one of the m trains that will depart from Shinjuku Station. The trains in Tokyo are extremely reliable, and Audrey knows to the second when each train will arrive at or depart from Shinjuku Station.

Audrey is a bit nervous about transferring at Shinjuku Station, so she does not want to have a tight transfer that takes *strictly* fewer than s seconds. However, neither does Audrey want to spend too long transferring at Shinjuku Station. Help Audrey determine the minimum transfer time possible. Audrey will not consider any itineraries that cross the boundary of a calendar day.

Input

The first line of input contains two integers n and m ($1 \le n, m \le 50$), the number of trains entering and leaving Shinjuku Station, respectively.

The next n lines each contain a string of the form `HH:MM:SS`, indicating the exact arrival time of a train entering Shinjuku.

The next m lines each contain a string of the form `HH:MM:SS`, indicating the exact departure time of a train leaving Shinjuku.

It is guaranteed that each of these times is a valid time. Specifically, `HH` is an integer between 5 and 23 inclusive, and both `MM` and `SS` are non-negative integers between 0 and 59. All values are represented using two digits, possibly with a leading zero.

The last line contains a single integer s ($1 \le s \le 600$), representing the minimum transfer time in seconds required by Audrey.

Output

Output a single integer, the minimum number of seconds that Audrey needs for a valid transfer. If no such transfer exists, output `-1`.

Example

Input	Output
2 2 10:00:00 11:00:00 10:05:00 11:04:00 300	300
1 1 23:13:23 13:23:13 23	-1

Problem H. Rows of Stars

Source file name:	Stars.c, Stars.cpp, Stars.java, Stars.py
Input:	`Standard`
Output:	`Standard`

You're an amateur astronomer! Lately, you've been studying one particular sequence of n stars in the sky observed at n spots. Each star has a fixed brightness value, but their positions rotate counterclockwise by k spots each day and wrap around.

For example, if the original sequence of stars had brightness values $[1, 2, 3, 4, 5]$ on day 1 and $k = 2$, then on day 2 the sequence would appear as $[3, 4, 5, 1, 2]$. The image below illustrates how the stars would appear over the next $n = 5$ days.

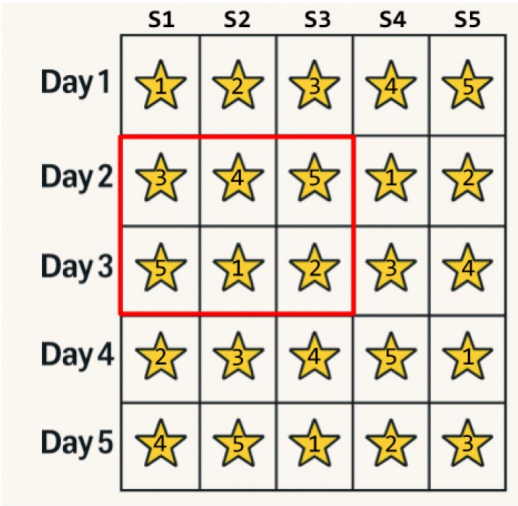

Figure 7. Illustration of the first example case. The stars at spots $[1, 3]$ over days $[2, 3]$ have a maximum sum of brightness of 20.

You believe something good will happen on the days when the stars are brightest. Thus, you are planning a d-day trip to observe the stars from s consecutive spots. You want to choose the spots and days so that the sum of brightness of the stars at those spots over those days is maximized. In other words, you want to find the largest possible sum of brightness of the stars at any s consecutive spots across any d consecutive days over the next n days. Note that the s spots you choose cannot wrap around.

Input

The first line of input contains four integers n, k, d, and s ($1 \le n \le 2 \cdot 10^5,\ 0 \le k \le 10^9,\ 1 \le d, s \le n$).

The next n lines each contain an integer between -10^7 and 10^7, inclusive. The integer on the ith line is the brightness value of the ith star on the first day.

Output

Output a single integer, the largest possible sum of brightness of the stars at any s consecutive spots across any d consecutive days over the next n days.

Example

Input	Output
5 2 2 3 1 2 3 4 5	20
4 2 3 2 6 -2 8 1	22
5 2 4 4 -1 3 -5 -7 -9	-58
1 1 1 1 1	1

Problem I. Manhattan Interference

Source file name:	Manhattan.c, Manhattan.cpp, Manhattan.java, Manhattan.py
Input:	`Standard`
Output:	`Standard`

In LOneLand, citizens have many options for telecom companies. Due to government regulations, each telecom company must have exactly two cell phone towers. A cell phone registered with a company will find the closest tower by Manhattan distance (also known as the L1-norm) and connect to that tower. The Manhattan distance between two points $(x_1,\ y_1)$ and $(x_2,\ y_2)$ is $|x_1 - x_2| + |y_1 - y_2|$.

If a cell phone is equidistant from both of its cell towers by Manhattan distance, then the device experiences *Manhattan Interference* and cannot connect to either tower. To improve the reliability of their phones, the citizens of LOneLand equip their devices with SIM cards from two different companies. Unfortunately, if a cell phone is ever at a point where both SIM cards experience Manhattan interference, the phone will explode!

Your task is to count the number of pairs of telecom companies for which equipping SIM cards from both companies is dangerous. That is, count the number of pairs of telecom companies for which there exists at least one point (not necessarily with integer coordinates) where a phone equipped with SIM cards from both companies would explode. Two pairs are considered different if there is a company in one pair that is not in the other.

Input

The first line of input contains a single integer n $\left(2 \le n \le 1.5 \cdot 10^5\right)$, the number of telecom companies.

Each of the next n lines contains four integers $x_1,\ y_1,\ x_2,\ y_2$ $\left(-10^8 \le x_1, y_1, x_2, y_2 \le 10^8\right)$, representing the coordinates of the two towers of one telecom company located at $(x_1,\ y_1)$ and $(x_2,\ y_2)$. All tower locations are distinct.

Output

Output a single integer, the number of pairs of telecom companies for which equipping SIM cards from both companies is dangerous.

Example

Input	Output
3 0 0 1 3 10 10 11 13 4 5 9 4	2

Explanation

There are two pairs of telecom companies that should be counted:

1. Companies 1 and 3: A phone will explode at point $(6,\ 1)$ if it connects to both companies, because it is 7 units away from both towers of company 1 and 6 units away from both towers of company 3.
2. Companies 2 and 3: A phone will explode at point $(7,\ 12)$ if it connects to both companies.

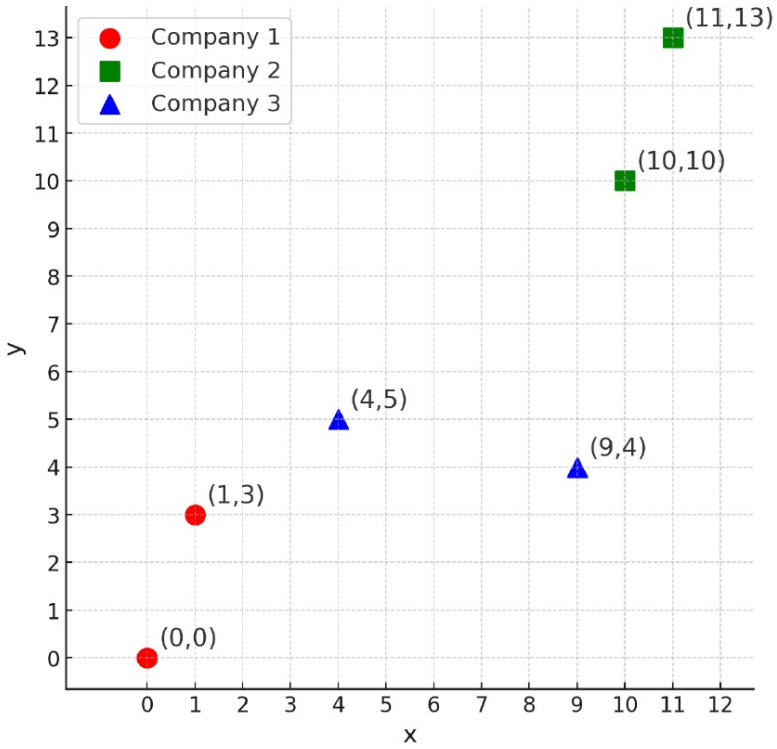

Figure 8. Illustration of the example case.

Problem J. Game of Nines

Source file name:	Game.c, Game.cpp, Game.java, Game.py
Input:	`Standard`
Output:	`Standard`

You are playing a simple game of adding digits. You are given a list of single digits between 0 and 8 (inclusive). In each move, you may choose any two digits a and b, add a to b, and replace b with the sum $a+b$. If $a+b \geq 10$, keep only the units digit (e.g. $5+8$ becomes 3; $4+6$ becomes 0). Whenever the sum is 9, the result is eliminated immediately and cannot participate in further additions.

Your goal is to eliminate as many digits as possible, using any number of moves.

Input

The first line of input contains a single integer n ($2 \leq n \leq 1000$), the number of digits. Each of the next n lines contains a single digit between 0 and 8, forming the initial list of digits.

Output

Output a single integer, the minimum number of digits that will remain if you play optimally to eliminate as many digits as possible.

Example

Input	Output
3 2 3 6	1
2 4 4	2

Explanation

Example Case 1: Add 3 to 6 and eliminate one digit ($3+6$ becomes 9). The remaining digits are 2 and 3. Then add 2 to 3 three times ($3+2$ becomes 5, $5+2$ becomes 7, $7+2$ becomes 9). The single digit remaining is 2. It is also possible to eliminate two digits with a different sequence of moves.

Problem K. Swap for Palindrome

Source file name:	Palindrome.c, Palindrome.cpp, Palindrome.java, Palindrome.py
Input:	`Standard`
Output:	`Standard`

Eric is a cool kid who just learned that a palindrome is a string that reads the same forward and backward. Recently, he found a string consisting of n lowercase English letters. He immediately started looking for palindromic substrings in it (a substring is a contiguous sequence of characters within a string). He soon realized that the string did not contain very long palindromic substrings.

Eric wants to improve the situation by performing exactly one swap of two letters in the string (the letters must be at different positions). What is the length of the longest palindromic substring that can be obtained after Eric performs exactly one swap?

Input

The input consists of a single string containing at least 2 and at most 5000 lowercase English letters (`a-z`).

Output

Output a single integer, the length of the longest palindromic substring that Eric can obtain after exactly one swap.

Example

Input	Output
`ccbaada`	`5`
`ab`	`1`

Explanation

Eample Case 1: Eric can swap the first `c` with the first `a`, i.e., `ccbaada`. The resulting string is `acbcada`, which has a palindromic substring of length 5: `acbca`.

Problem L. Tree Racing

Source file name:	Racing.c, Racing.cpp, Racing.java, Racing.py
Input:	`Standard`
Output:	`Standard`

It's time for one of the biggest high-stakes racing competitions of the year - the International Cutthroat and Professional Car-racing competition, also known as the ICPC.

This year's racetrack is a connected network of n checkpoints, numbered from 1 to n. These checkpoints are connected by $n-1$ tunnels of equal unit length such that every checkpoint is reachable from every other checkpoint. Before the race begins, racers will spawn at various checkpoints throughout the racetrack, and their goal is to race to the finish checkpoint via the shortest path.

Certain checkpoints on the racetrack are *special*, which allow only k racers to pass through. In other words, only the fastest k racers can pass each such checkpoint, while the rest who attempt to pass are eliminated from the race. If multiple racers reach a special checkpoint at the exact same time, those with higher speeds are allowed to pass first. A racer who spawns at a special checkpoint is considered to have immediately passed that checkpoint, which counts towards the k racers.

As an avid fan of the ICPC, you know the speed of each racer's vehicle in this year's event, and that each racer's vehicle speed remains constant throughout the race. You want to predict the leaderboard by calculating how long it will take each racer to reach the finish checkpoint.

Input

The first line of input contains three integers n, m, and k ($2 \le n \le 2 \cdot 10^5,\ 1 \le m \le n-1,\ 1 \le k \le 10$), where n is the number of checkpoints within the racetrack, m is the number of racers, and k is the number of racers that can pass through a special checkpoint.

The next $n-1$ lines each contain two integers a and b ($1 \le a, b \le n,\ a \ne b$), denoting a tunnel connecting checkpoints a and b.

The next m lines each describe a racer. The ith line has two integers p and t $\left(1 \le p \le n,\ 1 \le t \le 10^9\right)$, indicating that racer i spawns at checkpoint p with a vehicle that takes t seconds to travel through a unit-length tunnel. No two racers spawn at the same checkpoint. No two racers have the same vehicle speed, and all t values are distinct.

The next line contains a single integer e ($1 \le e \le n$), denoting the finish checkpoint. No racer spawns at the finish checkpoint.

The next line contains a single integer c ($1 \le c \le n-1$), the number of special checkpoints.

The next c lines each contain one integer x ($1 \le x \le n,\ x \ne e$), denoting that checkpoint x is a special checkpoint. All c special checkpoints are unique.

Output

Output m lines. On the ith line, output the number of seconds required for racer i to reach the finish checkpoint. If racer i will be blocked by a special checkpoint and eliminated, output `-1`.

Example

Input	Output
8 5 2 2 1 1 3 4 5 1 4 4 6 6 7 6 8 5 2 3 4 6 3 7 1 8 5 2 2 1 6	6 -1 -1 4 -1

Explanation

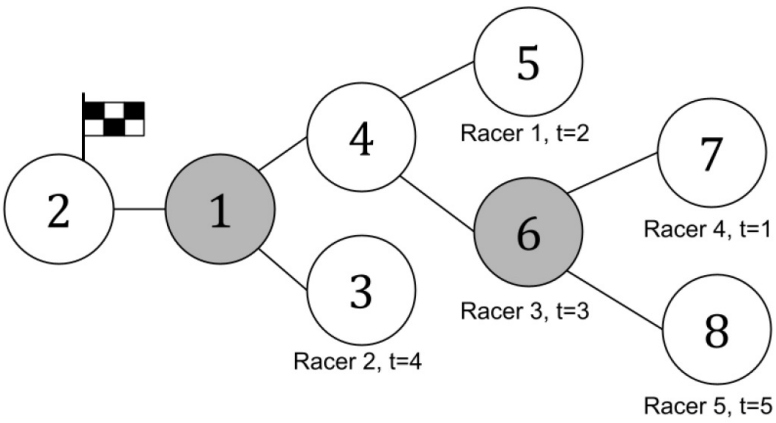

Figure 9. Illustration of the example case. Checkpoint 2 is the finish checkpoint. Checkpoints 1 and 6 are special (shown in gray).

Problem M. Much Room for Mushrooms

Source file name:	Mushrooms.c, Mushrooms.cpp, Mushrooms.java, Mushrooms.py
Input:	`Standard`
Output:	`Standard`

When botany proves too challenging, you can always switch to *mycology* - the study of mushrooms! As one of your first experiments, you plan to grow many mushrooms inside an infinitely large 2D exhibit, which can be modeled as a grid of tiles.

As it turns out, this specific species of mushroom that you are growing has a consistent shape. Each mushroom can be represented as a vertical stem (of any height) that is exactly one tile wide. Then, the very top tile in the stem juts outwards by one tile to the left, right, and up to form the cap of the mushroom. The figure below shows three such mushrooms:

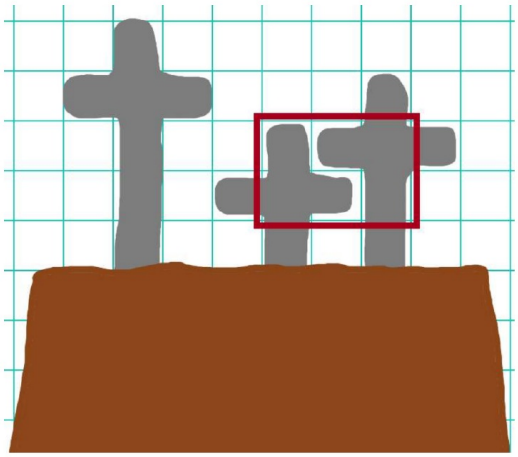

Figure 10. A visualization of the first example case.

You've noticed that for each mushroom to survive, two conditions must be met in your exhibit:

1. No tile can be shared by more than one mushroom.
2. Every mushroom must begin growing on the same row.

To make your exhibit more interesting, you'd like to grow mushrooms so that there exists an $r \times c$ rectangle in your exhibit where every tile is occupied by a mushroom. But in this exhibit, there is *not much room for mushrooms*. What is the minimum number of mushrooms needed to completely fill an $r \times c$ rectangle with mushrooms, assuming you can control the position and height of each mushroom you plant?

Input

The first line of input contains two integers r and c ($1 \leq r, c \leq 1000$), the number of rows and columns of the rectangle in your exhibit you want to completely fill with mushrooms.

Output

Output a single integer, the minimum number of mushrooms that are needed to completely fill an $r \times c$ rectangle, or `-1` if it is not possible.

Example

Input	Output
2 3	2
4 1	1
100 100	-1